EXPERIMENT 7

Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

The dataset used for this experiment is:

Churn Modelling Dataset

Source Platform:
Kaggle

Dataset Link:
https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

The Churn Modelling dataset is used to predict whether a customer will leave a bank (churn) or not.

Dataset Size:

- Number of records: 10,000
- Number of features: 10 input features (after preprocessing)
- Target variable: 1

Target Variable:

`Exited`

- 0 → Customer stays
- 1 → Customer leaves (churn)

Feature Description:

Feature	Description

CreditScore	Credit score of the customer
Geography	Country of the customer
Gender	Male or Female
Age	Age of the customer
Tenure	Number of years with bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Has credit card (0/1)
IsActiveMember	Active member (0/1)
EstimatedSalary	Estimated salary
Exited	Target variable

Dataset Characteristics:

- Binary classification problem
- Contains categorical and numerical features
- Requires preprocessing (encoding + scaling)

3. Mathematical Formulation of the Algorithm

Artificial Neural Network (ANN) is inspired by biological neurons and consists of layers:

- Input Layer
- Hidden Layer(s)
- Output Layer

Neuron Computation:

Z=∑i=1$nwixi$+bZ=i=1∑nwixi+bA=f(Z)A=f(Z)

Where:

- xixi = input features
- wiwi = weights
- bb = bias
- $f(Z)$f(Z) = activation function

Activation Functions:

ReLU:

$f(x)$=max(0,x)f(x)=max(0,x)

Sigmoid:

$f(x)$=11+e−xf(x)=1+e−x1

Loss Function (Binary Cross Entropy):

$Loss$=−1N∑[$y$log(y^)+(1−y)log(1−y^)]Loss=−N1∑[ylog(y^)+(1−y)log(1−y^)]

Optimization:

Weights are updated using **Gradient Descent**:

w=w−η∂L∂ww=w−η∂w∂L

Where:

- ηη = learning rate

4. Algorithm Limitations

- Requires large amount of data for good performance
- Computationally expensive
- Prone to overfitting
- Requires hyperparameter tuning
- Less interpretable compared to traditional models
- Training time is high

5. Methodology / Workflow

Step 1: Data Collection

Dataset downloaded from Kaggle.

Step 2: Data Preprocessing

- Removed irrelevant columns (RowNumber, CustomerId, Surname)
- Encoded categorical variables (Gender, Geography)
- Applied feature scaling using StandardScaler

Step 3: Train-Test Split

- 80% training data
- 20% testing data

Step 4: Build ANN Model

- Input layer
- Hidden layers using ReLU activation
- Output layer using Sigmoid activation

Step 5: Model Training

- Used binary cross-entropy loss
- Optimizer: Adam
- Trained over multiple epochs

Step 6: Model Evaluation

- Accuracy
- Confusion matrix

Workflow Diagram

Dataset
↓
Data Cleaning
↓
Encoding + Scaling
↓

Train-Test Split
↓
ANN Model Building
↓
Training (Epochs)
↓
Prediction
↓
Evaluation

Code Implementation

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
df = pd.read_csv('Churn_Modelling.csv')

# Drop unnecessary columns
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)

# Encode categorical data
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

df = pd.get_dummies(df, columns=['Geography'], drop_first=True)

# Split features and target
X = df.drop('Exited', axis=1)
y = df['Exited']
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build ANN
model = Sequential()

model.add(Dense(units=16, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(units=8, activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=20, batch_size=32)

# Predictions
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Output

```
Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 4/13/2026 - 100% done
Saving Churn_Modelling.csv to Churn_Modelling (1).csv
Dataset Shape: (10000, 14)
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.7487 - loss: 0.5519 - val_accuracy: 0.7981 - val_loss: 0.4663
Epoch 2/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7962 - loss: 0.4500 - val_accuracy: 0.8131 - val_loss: 0.4320
Epoch 3/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8072 - loss: 0.4300 - val_accuracy: 0.8225 - val_loss: 0.4208
Epoch 4/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8164 - loss: 0.4176 - val_accuracy: 0.8200 - val_loss: 0.4116
Epoch 5/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8238 - loss: 0.4061 - val_accuracy: 0.8281 - val_loss: 0.4008
Epoch 6/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8416 - loss: 0.3893 - val_accuracy: 0.8419 - val_loss: 0.3842
Epoch 7/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8497 - loss: 0.3737 - val_accuracy: 0.8475 - val_loss: 0.3736
Epoch 8/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8533 - loss: 0.3645 - val_accuracy: 0.8487 - val_loss: 0.3669
Epoch 9/20
```

```
Epoch 9/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8530 - loss: 0.3592 - val_accuracy: 0.8481 - val_loss: 0.3626
Epoch 10/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8542 - loss: 0.3554 - val_accuracy: 0.8462 - val_loss: 0.3592
Epoch 11/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8566 - loss: 0.3529 - val_accuracy: 0.8481 - val_loss: 0.3567
Epoch 12/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8545 - loss: 0.3512 - val_accuracy: 0.8469 - val_loss: 0.3549
Epoch 13/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8562 - loss: 0.3500 - val_accuracy: 0.8481 - val_loss: 0.3538
Epoch 14/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8570 - loss: 0.3482 - val_accuracy: 0.8469 - val_loss: 0.3515
Epoch 15/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8556 - loss: 0.3466 - val_accuracy: 0.8494 - val_loss: 0.3507
Epoch 16/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8572 - loss: 0.3460 - val_accuracy: 0.8519 - val_loss: 0.3510
Epoch 17/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8570 - loss: 0.3454 - val_accuracy: 0.8487 - val_loss: 0.3499
Epoch 18/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8562 - loss: 0.3441 - val_accuracy: 0.8487 - val_loss: 0.3486
Epoch 19/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8580 - loss: 0.3437 - val_accuracy: 0.8481 - val_loss: 0.3474
Epoch 20/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8572 - loss: 0.3426 - val_accuracy: 0.8475 - val_loss: 0.3463
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Accuracy: 0.8625

Confusion Matrix:
 [[1559   48]
 [ 227  166]]

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.97      0.92      1607
           1       0.78      0.42      0.55       393

    accuracy                           0.86      2000
   macro avg       0.82      0.70      0.73      2000
weighted avg       0.85      0.86      0.85      2000
```

6. Performance Analysis

The model achieved satisfactory accuracy in predicting customer churn.

Observations:

- ANN successfully captured non-linear relationships
- Accuracy improved with training epochs
- Model correctly classified majority of customers

Interpretation:

- High accuracy indicates good learning
- Some misclassifications exist due to overlapping features

7. Hyperparameter Tuning

Parameters tuned:

- Number of hidden layers
- Number of neurons
- Activation functions
- Batch size
- Number of epochs

Example improvements:

- Increasing neurons improved learning capacity
- More epochs improved accuracy but risked overfitting
- Adam optimizer provided stable convergence

Tuning resulted in better model performance and reduced error.

8. Conclusion

- Artificial Neural Network was successfully implemented for customer churn prediction.
- The model effectively handled both numerical and categorical data after preprocessing.
- ANN provided good accuracy and captured complex patterns in data.
- Proper hyperparameter tuning improved performance.
- ANN is a powerful model for classification problems involving large datasets.